Submitted by:

Shehriyar khan

Submitted to:

Ma'am Zeenat Zulfiqar

Course title:

Artificial Intelligence

Registration no:

SP23-BSE-162

Deadline:

15 April 2026

LAB TASK 02:

Task 01 Description:

1. Create a Python program to instantiate a specified class and display the namespace of that instance.
2. Define a function student data() that prints the student ID. Extend the function to optionally display student name and class when provided.
3. Create a class named Student and examine its type. Display the keys of its dict attribute and the value of its module .
4. Create two empty classes Student and Marks. Instantiate objects and verify: • Whether the objects belong to their respective classes • Whether these classes are derived from the built-in object class
5. Define a class Student with attributes student name and marks. Modify attribute values and display both original and updated values.
6. Create a class Student with attributes student id and student name. Add a new attribute student class, display all attributes, then remove student name and display the updated attributes.

CODES:

Question 01

[1]
0s

```python
# Task 1
class MyClass:
    def __init__(self):
        self.a = 10
        self.b = 20

obj = MyClass()
print("Instance data:", obj.__dict__)
```

```
Instance data: {'a': 10, 'b': 20}
```

Question 02

[2]
0s

```python
# Task 2
def student_data(id, name=None, clas=None):
    print("ID:", id)
    if name != None:
        print("Name:", name)
    if clas != None:
        print("Class:", clas)

student_data(1)
student_data(2, "Ali", "BSCS")
```

```
ID: 1
ID: 2
Name: Ali
Class: BSCS
```

Question 03

[3]
✓ 0s

```python
# Task 3
class Student:
    pass

print(type(Student))
print(Student.__dict__.keys())
print(Student.__module__)
```

```
<class 'type'>
dict_keys(['__module__', '__dict__', '__weakref__', '__doc__'])
__main__
```

Question 04

[4]
✓ 0s

```python
# Task 4
class Student:
    pass

class Marks:
    pass

s = Student()
m = Marks()

print(isinstance(s, Student))
print(isinstance(m, Marks))

print(issubclass(Student, object))
print(issubclass(Marks, object))
```

```
True
True
True
True
```

Question 05

[6]
✓ 0s

```python
# Task 5
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

s = Student("Ali", 80)

print("Before:", s.name, s.marks)

s.name = "Ahmed"
s.marks = 90

print("After:", s.name, s.marks)
```

```
Before: Ali 80
After: Ahmed 90
```

Question 06

[7]
✓ 1s

```python
# Task 6
class Student:
    def __init__(self, id, name):
        self.id = id
        self.name = name

s = Student(1, "Ali")

s.clas = "BSCS"
print("Before delete:", s.__dict__)

del s.name
print("After delete:", s.__dict__)
```

```
Before delete: {'id': 1, 'name': 'Ali', 'clas': 'BSCS'}
After delete: {'id': 1, 'clas': 'BSCS'}
```

Question 07

[8]
✓ 0s

```python
# Task 7
class Student:
    def __init__(self, id, name, clas):
        self.id = id
        self.name = name
        self.clas = clas

    def show(self):
        print("ID:", self.id)
        print("Name:", self.name)
        print("Class:", self.clas)

s = Student(1, "Ali", "BSCS")
s.show()
```

```
ID: 1
Name: Ali
Class: BSCS
```

Question 08

[9]
0s

```python
# Task 8
s1 = Student(1, "Ali", "BSCS")
s2 = Student(2, "Sara", "BBA")

print("Student 1")
s1.show()

print("Student 2")
s2.show()
```

```
Student 1
ID: 1
Name: Ali
Class: BSCS
Student 2
ID: 2
Name: Sara
Class: BBA
```

Question 09 & 10

[10]
✓ 0s

```python
# Task 9 & 10 (simple)
import sys

print("Memory:", sys.getsizeof(s1))

class Student2:
    __slots__ = ['id', 'name', 'clas']

    def __init__(self, id, name, clas):
        self.id = id
        self.name = name
        self.clas = clas

s3 = Student2(1, "Ali", "BSCS")
print("Memory with slots:", sys.getsizeof(s3))
```

```
Memory: 48
Memory with slots: 56
```

Task 01 Description:

1. Design two classes representing Trapezoid and Parallelogram. Define appropriate attributes and compute their areas using standard mathematical formulas.
2. Develop a comparison mechanism to evaluate areas of different shapes.
3. Store the computed areas in a dictionary structure.
4. Iterate over multiple sets of dimensions to analyze variations in area.
5. Utilize dictionary operations to determine and display the shape with the maximum area.

Code:

Question 01

[1]
0s

```python
# Class for Trapezoid
class Trapezoid:
    def __init__(self, a, b, h):
        self.a = a
        self.b = b
        self.h = h

    def area(self):
        return (self.a + self.b) * self.h / 2


# Class for Parallelogram
class Parallelogram:
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        return self.base * self.height


# Different sets of dimensions
t1 = Trapezoid(10, 8, 5)
t2 = Trapezoid(6, 4, 3)

p1 = Parallelogram(12, 6)
p2 = Parallelogram(10, 5)
```

```python
# Store areas in dictionary
areas = {}

areas["Trapezoid1"] = t1.area()
areas["Trapezoid2"] = t2.area()
areas["Parallelogram1"] = p1.area()
areas["Parallelogram2"] = p2.area()


# Display all areas
print("Areas of shapes:")
for shape in areas:
    print(shape, "=", areas[shape])


# Find shape with maximum area
max_shape = max(areas, key=areas.get)

print("\nShape with maximum area:", max_shape)
print("Maximum area:", areas[max_shape])
```

```
Areas of shapes:
Trapezoid1 = 45.0
Trapezoid2 = 15.0
Parallelogram1 = 72
Parallelogram2 = 50

Shape with maximum area: Parallelogram1
Maximum area: 72
```